

离散计算机算术（DCA）：面向有限计算的整理稿

作者：王秉钦 单位：北京国家会计学院 版本：2026-07-04

摘要

本文把“离散计算机算术”（Discrete Computer Arithmetic）简称为 DCA。这里的 DCA 不是要替代现有数学，也不是要宣称一个更大的数学体系，而是给计算机运算建立一套低调、可实现、便于检查的表达方式。本文只关心一件事：当对象最终要落到程序、芯片、证明助手或有限数据结构中时，怎样把定义、运算和论证过程写得足够明确。

本文的基本态度是：连续对象可以作为外部规格、极限直觉或误差分析工具，但实现层只直接处理有限编码、整数、有理数、有限图、有限矩阵、有限状态机和有限证明对象。这样做的好处是定义边界清楚，证明过程更容易机械检查，代码也更容易复现。

统一约定

1. 对象约定：本文中的可计算对象都应当有有限编码，记为 $\text{enc}(x) \in \{0,1\}^*$ 。整数、有限列表、矩阵、图、程序和证明脚本都属于此类。
2. 边界约定：任何算法都必须说明字长、模数、状态空间、循环上界或终止条件。若使用无限集合，只把它当作外部规格或生成规则，不当作一次性存储对象。
3. 算术约定：机器字长下的加法默认是模 2^w 的回绕运算；若需要饱和、截断、任意精度或有理数，应显式声明。
4. 证明约定：本文常用四类论证：有限枚举、结构归纳、循环不变式、代数恒等式化简。对有限域或有限集合上的命题，穷举验证也可以是有效证明。
5. 近似约定：涉及实数、复数、微分、概率密度、量子振幅等概念时，本文只给出离散表示或整数缩放表示，并保留误差界或适用条件。

调研资料

整理过程中参考了若干基础资料：Turing 的可计算性论文、Shannon 的通信理论、Cooley 与 Tukey 的 FFT 论文、离散外微分与离散微分几何资料、Forman 的离散 Morse 理论、Rocq/Coq 归纳类型文档、RISC-V 指令集文档、整数神经网络量化论文、NIST 后量子密码标准，以及元胞自动机普适性相关文献。它们在文末列为参考资料。本文只吸收其中与有限表示、离散结构和可实现算法相关的部分。

已有工作与实例

进一步调研后可以确认，本文许多部分并不是空想中的方向，而是已经散落在计算数学、形式化验证、密码学、优化、图算法、机器学习部署、操作系统和硬件设计里。下面按 43 个部分列出可参考实例。它们只是“已有实现或研究线索”，不表示这些项目都采用本文的 DCA 名称。

1. 算术基础：大整数与模算术已有成熟实现，如 [GMP](#)、[FLINT](#) 和 SageMath 的整数/有限域接口。密码实现里，[Fiat-Crypto](#) 已经把有限域算术从 Coq/Rocq 证明生成到 C 代码；[HACL*](#) 和 EverCrypt 则把高保证密码原语落到可部署 C/汇编。
2. 代数结构：[GAP](#) 专门面向计算离散代数，尤其是计算群论；[SageMath](#) 把 GAP、FLINT、NumPy、SciPy、Singular 等系统整合为开放数学计算环境；[arkworks algebra](#) 在 Rust 中实现 zkSNARK 所需的有限域、椭圆曲线和多项式。
3. 离散分析：有限差分、有限元和有限体积方法已经是科学计算主线。[SciPy](#) 和 [PETSc](#) 提供稀疏矩阵、差分方程求解与大规模数值计算基础；图和网格上的离散拉普拉斯则在 [libigl](#)、[geometry-central](#)、[GUDHI](#) 等库中直接使用。

4. 离散几何：[libigl](#) 提供稀疏离散微分几何算子、cotangent Laplacian 和网格拓扑结构；[geometry-central](#) 以曲面网格为中心，强调只依赖算法真正需要的几何数据，如边长、内蕴几何和邻接结构。
5. 逻辑与推理：SAT/SMT 求解器已经是软件验证和硬件验证的基础工具。[Kissat](#)、[CaDiCaL](#)、[Z3](#) 和 [cvc5](#) 都是可直接复用的实例。
6. 离散傅里叶分析与 NTT：数论变换已经广泛用于多项式乘法、zk 证明和后量子密码。2025 年 IACR Communications in Cryptology 出现了形式化验证 NTT 核心操作的工作；Winterfell、Plonky2 和 arkworks 也都在有限域 FFT/FRI/多项式承诺中使用类似结构。
7. 离散概率论：有限样本空间、计数概率和离散随机变量在概率编程与统计软件中已有大量实现。[PyMC](#)、[Stan](#) 和 [TensorFlow Probability](#) 支持离散分布；通信仿真库 Sionna 和 FEC 工具 AFF3CT 也大量使用离散概率与有限块长仿真。
8. 离散微分几何：离散外微分和离散曲率已经形成工程工具链。[Discrete Differential Geometry](#) 课程资料、[libigl](#)、[geometry-central](#) 和 2026 年的 [Dxtr](#) 都是可参考实例。
9. 离散动力系统与整数 AI 运算：整数推理已进入主流部署。[LiteRT/TensorFlow Lite 8-bit quantization spec](#)、[ONNX Runtime quantization](#) 和 PyTorch 量化工具都把神经网络推理落到 int8/int16/定点算术。
10. 离散复分析与二元数：复数和对偶数在自动微分系统中以程序对象出现。JAX、PyTorch、TensorFlow 的自动微分都可看作有限计算图上的链式法则实现；有限域扩张则在 arkworks、SageMath、FLINT 和 zk 证明系统中大量使用。
11. 离散微分方程：工程上实际执行的方程求解多为离散递推。[SciPy integrate](#)、[DifferentialEquations.jl](#) 和 [do-mpc](#) 都把连续模型离散成有限步迭代或有限时域优化。
12. 离散优化与控制：[Google OR-Tools](#) 提供 CP-SAT、整数规划、图算法和调度工具；[JuMP](#) 支持整数规划、锥规划和非线性优化；[CVXPY](#) 也包含混合整数问题接口。
13. 离散信息论与编码：[AFF3CT](#) 支持 Turbo、LDPC、Polar 等前向纠错码仿真；[NVIDIA Sionna](#) 面向 5G/6G 通信链路、系统级仿真和机器学习通信；5G NR 中的 LDPC 与 Polar code 是已经标准化的工程实例。
14. 从数学定义到指令集：[RISC-V ISA manual](#) 展示了开放指令集如何精确定义整数指令、异常和状态转移；[Sail](#) 可用于写 ISA 语义；CompCert、CakeML 和 seL4 的证明链也都依赖精确机器模型。
15. 离散拓扑与组合调谐：[GUDHI](#)、[Ripser](#)、[Dionysus](#) 和 [giotto-tda](#) 已经把单纯复形、持久同调和拓扑机器学习做成可用软件。
16. 有限域上的代数几何：[Macaulay2](#) 和 [Singular](#) 是代数几何、交换代数和多项式计算的经典系统；SageMath 则把有限域曲线、编码、椭圆曲线和多项式理想整合到统一环境。
17. 构造数学与类型论：[Rocq Prover](#)、[Lean 4 / mathlib](#)、[Isabelle](#) 和 [Dafny](#) 都在用类型、归纳定义、规格和证明对象表达可检查数学与程序。
18. 离散自动微分：JAX、PyTorch、TensorFlow、Enzyme、MLIR autodiff 等都把程序转化为有限计算图或中间表示上的导数传播。整数训练仍需近似梯度，但整数推理、定点反传和量化感知训练已经有工业工具支持。
19. 从定义到形式化验证：[Project Everest](#)、HACL*、Fiat-Crypto、[seL4](#)、[CompCert](#) 和 [CakeML](#) 都是“从规格到实现”的成熟案例。
20. 离散随机过程与鞅：二叉树期权定价、有限马尔可夫链、排队网络、离散时间金融模型都已有工具支持。QuantLib、Sionna、NumPy/SciPy 和概率编程系统可以作为实现参考。
21. 离散度量空间：[NetworkX](#) 提供最短路、连通性、中心性、聚类等图度量算法；Annoy、FAISS、hnsplib 等近邻库则把高维距离搜索工程化，虽然有些内部使用浮点，但索引结构本身是离散图或树。
22. 有限维算子代数：稀疏矩阵、图拉普拉斯、有限维线性算子在 SciPy、SuiteSparse、Eigen、PETSc、GraphBLAS 中都有实现。[SuiteSparse:GraphBLAS](#) 特别接近“代数结构 + 图算法”的路线。
23. 离散信号处理：SciPy Signal、FFTW、KFR、LiquidDSP、GNU Radio 等都实现了 FIR/IIR、抽取、插值、FFT 和通信信号处理。整数小波和 lifting scheme 在 JPEG 2000 等标准中已有工程应用。
24. DCA-ISA 与微架构草图：RISC-V 自定义指令、MLIR/IREE 编译栈、TVM、XLA、Triton 与各类 NPU/TPU 编译器都在做“从张量/多项式/循环到低层机器操作”的映射。[IREE](#) 明确面向从数据中心到移动和嵌入式部署。

25. 操作系统与认证内核：seL4 是最清晰实例，已有机器检查的内核正确性证明，并在 Arm、RISC-V、Intel 等架构上维护证明栈。TLA+ 与 TLC 也常用于分布式系统和 OS 协议的有限状态建模。
26. 有限域量子模型：主流量子软件如 Qiskit、Cirq、QuTiP 5 和 Stim 都把量子线路、稳定子电路或开放系统仿真变成有限数据结构和有限步计算。
27. 离散时空与因果元胞自动机：格点场论、有限差分域法、Lattice Boltzmann 方法和元胞自动机都属于已有实例。Golly 可直接探索生命游戏、von Neumann 自动机、WireWorld、Langton loops 等规则。
28. 计算复杂性：复杂性理论本身以有限输入串和资源界为对象。SAT Competition、SMT-COMP、DIMACS 格式、QBF 求解器、模型检查 benchmark 都是把理论问题变成可运行实例的做法。
29. 元胞自动机与计算普适性：Golly 是开放的元胞自动机探索工具；Rule 110 普适性、生命游戏图灵机、WireWorld 逻辑门都是“局部离散规则实现计算”的经典实例。
30. 形式化验证闭环：TLA+ 工具链、Dafny、SPARK Ada、Frama-C、Why3、Kani Rust Verifier 都是规格-验证-实现闭环的工程工具。Kani 对 Rust 做位精确模型检查，尤其贴有限状态验证。
31. 离散信息几何：有限分布、总变差距离、KL 的定点近似、离散 Fisher 信息在机器学习和统计推断里都已实现。scikit-learn、PyTorch、JAX 与概率编程系统可作为实验平台。
32. 符号动力学与离散混沌：子移位、有限自动机、模整数 cat map 和伪随机置乱常见于图像加密、符号序列分析和动力系统教学。NetworkX 可用于把有限型子移位转成有向图路径问题。
33. 离散最优控制：有限时域动态规划、MPC、混合整数 MPC 已有成熟工具。do-mpc、CasADi、JuMP、OR-Tools 和 cvxpy 都能表达不同层次的离散或离散化控制问题。
34. 代数编码与密码学：NIST 2024 年发布的 FIPS 203 ML-KEM、FIPS 204 ML-DSA、FIPS 205 SLH-DSA 是近年最重要的后量子密码工程实例；zk 系统中的 arkworks、Winterfell、Plonky2 也依赖有限域、多项式和编码思想。
35. DCA 的表达范围：计算数学软件生态本身就是范围证据。SageMath、GAP、FLINT、Singular、Macaulay2、NetworkX、OR-Tools、GUDHI 等覆盖了离散代数、数论、优化、图和拓扑。
36. 物理实现蓝图：RISC-V、CHERI、seL4、MLIR/IREE、TVM、开源 EDA 工具和形式化硬件验证都在把高层规格落到机器结构。这里更适合写作“实现路线图”，而不是宣称一种新物理。
37. 离散微分拓扑与 Morse 理论：Forman 离散 Morse 理论已经有多年发展；GUDHI、Dionysus、giotto-tda 和相关 TDA 工具把临界单形、过滤和持久同调用于数据分析。
38. 离散谱理论：图拉普拉斯、谱聚类、PageRank、随机游走和图神经网络都建立在有限矩阵上。NetworkX、PyTorch Geometric、DGL、SciPy sparse 是常见实现平台。
39. 离散神经网络架构搜索：NAS-Bench 系列、NNI、Optuna、Ray Tune 和 TVM auto-scheduler 都把架构、算子选择、延迟和内存预算转化为有限搜索或优化问题。
40. DCA 自举解释器：自解释器、字节码 VM、WebAssembly、CakeML 自举编译器、Racket/Scheme 教学解释器都是可参考实例。CakeML 尤其适合作为“验证过的语言实现”参考。
41. 离散物理映射：格点规范理论、lattice Boltzmann、FDTD、电磁和弹性模拟、元胞自动机物理模型都已有软件与论文积累。本文宜把它们作为数值模型和计算模型实例。
42. 定理证明自动化：SAT/SMT、E-prover、Vampire、Lean 自动化、Rocq tactics、Isabelle Sledgehammer、cvc5 proof output 都属于可用路线。近年 LLM 也开始辅助 Lean/Isabelle 证明搜索，但仍需由证明器检查。
43. 全离散智能体概念：可审计智能体可以参考规划器、模型检查器、规则引擎、整数推理模型和安全约束系统。更现实的落点是“有限状态规划 + 量化模型 + 形式化安全壳”，而不是直接宣称 AGI。

目录

第1部分：算术基础 第2部分：代数结构 第3部分：离散分析 第4部分：离散几何 第5部分：逻辑与推理
第6部分：离散傅里叶分析与数论变换 第7部分：离散概率论 第8部分：离散微分几何 第9部分：离散动力系统与整数
AI 运算 第10部分：离散复分析与二元数 第11部分：离散微分方程 第12部分：离散优化与控制

第13部分：离散信息论与编码 第14部分：从数学定义到指令集 第15部分：离散拓扑与组合同调
第16部分：有限域上的代数几何 第17部分：构造数学与类型论 第18部分：离散自动微分
第19部分：从定义到形式化验证 第20部分：离散随机过程与鞅 第21部分：离散度量空间 第22部分：有限维算子代数
第23部分：离散信号处理 第24部分：DCA-ISA 与微架构草图 第25部分：操作系统与认证内核
第26部分：有限域量子模型 第27部分：离散时空与因果元胞自动机 第28部分：计算复杂性
第29部分：元胞自动机与计算普适性 第30部分：形式化验证闭环 第31部分：离散信息几何
第32部分：符号动力学与离散混沌 第33部分：离散最优控制 第34部分：代数编码与密码学 第35部分：DCA
的表达范围 第36部分：物理实现蓝图 第37部分：离散微分拓扑与 Morse 理论 第38部分：离散谱理论
第39部分：离散神经网络架构搜索 第40部分：DCA 自举解释器 第41部分：离散物理映射 第42部分：定理证明自动化
第43部分：全离散智能体概念

第1部分：算术基础

定义

设 $B=\{0,1\}$ 。一个 w 位无符号整数是一个比特串 $a=(a_{w-1},\dots,a_0) \in B^w$ ，其值为 $\text{val}(a)=\sum_{i=0}^{w-1} a_i 2^i$ 。机器加法定义为模 2^w 加法：

$$a+_wb=(\text{val}(a)+\text{val}(b)) \bmod 2^w。$$

逐位实现中，第 i 位和进位满足：

$$s_i=a_i \text{ xor } b_i \text{ xor } c_i, \quad c_{i+1}=\text{majority}(a_i,b_i,c_i)。$$

减法可以定义为补码加法： $a-_wb=a+_w((\sim b)+_w 1)$ 。乘法定义为有限个移位加法的和：

$$a*_wb=\sum_{i:b_i=1} (a \ll i) \bmod 2^w。$$

除法不应写成实数除法，而应返回商余对 (q,r) ，满足 $a=bq+r$ 且 $0 \leq r \ll b$ 。在机器字长内，若需要防止中间溢出，应使用更宽字长或任意精度整数。

论证过程

加法器的正确性可用位归纳证明。归纳不变式为：处理完低 k 位后，已经写入的结果等于 $(a \bmod 2^k + b \bmod 2^k) \bmod 2^k$ ，而 c_k 正是第 k 位进位。 $k=0$ 时成立；从 k 到 $k+1$ 时，异或给出当前位的奇偶，majority 给出至少两个输入为 1 时的进位，所以不变式保持。处理 w 位后得到模 2^w 的和。

乘法的正确性来自二进制展开： $b=\sum b_i 2^i$ ，所以 $ab=\sum b_i(a2^i)$ 。每个非零位只触发一次移位加法，因此过程有限。除法可用长除法不变式证明：从高位到低位试商时，始终保持 原始 $a=bq+r$ ，并保持 $r \geq 0$ ；最后若没有位还能继续扣除 b ，则 $r \ll b$ 。

实现要点

在程序里应明确三种语义：模回绕、饱和、任意精度。很多数学性质依赖语义选择。例如 w 位回绕加法构成阿贝尔群；饱和加法通常不满足结合律；任意精度整数满足普通整数环的性质。开源实现中建议把 $\text{Word}[w]$ 、 Nat 、 Int 、 $\text{Mod}[p]$ 分成不同类型，避免同一个 $+$ 隐含多套规则。

第2部分：代数结构

定义

有限集合是可枚举对象的无重复有限列表，通常记为 $A=\{a_0, \dots, a_{n-1}\}$ 。函数 $f:A \rightarrow B$ 在实现层可以表示为查表、程序或二者结合；若表示为程序，必须保证对 A 中每个输入有限步结束。

群是二元运算 $\ast:G \times G \rightarrow G$ 以及单位元 e 的结构，满足封闭性、结合律、单位元和逆元。有限环是带有加法和乘法的有限集合，满足加法群、乘法结合律和分配律。有限域是非零元素在乘法下也构成群的有限环。

典型例子是 $\mathbb{Z}/n\mathbb{Z}$ 。当 n 为素数时， $\mathbb{Z}/p\mathbb{Z}$ 是域；当 n 非素数时，存在零因子，通常不是域。 w 位机器整数在回绕加法和乘法下同构于 $\mathbb{Z}/2^w\mathbb{Z}$ ，它是有限交换环，但当 $w \geq 2$ 时不是域。

论证过程

有限代数结构的性质可以机械验证。给定 G 的乘法表，封闭性由表中元素是否仍在 G 内决定；结合律可枚举所有三元组 (a,b,c) 检查 $(a \ast b) \ast c = a \ast (b \ast c)$ ；单位元和逆元可通过有限搜索找到。有限域中每个非零元素的逆元存在，可用扩展欧几里得算法证明：若 $\gcd(a,p)=1$ ，则存在整数 u,v 使 $ua+vp=1$ ，于是 u 是 a 的模 p 逆元。

矩阵、向量和线性映射在这里都只取有限维。矩阵乘法的结合律来自有限求和的重新括号：

$$((AB)C)_{ij} = \sum_k (\sum_l A_{il} B_{lk}) C_{kj} = \sum_l A_{il} (\sum_k B_{lk} C_{kj}) = (A(BC))_{ij}。$$

若底层环是模环，所有等式在同一个模数下解释。

实现要点

代数结构应尽量用“载体集合 + 运算 + 性质检查器”表达。对小规模结构，可以直接枚举验证；对大规模有限域，则用算法证明替代全量枚举。矩阵库需要显式记录维度、底层环、模数和溢出策略，否则同一段代码可能在数学上属于不同结构。

第3部分：离散分析

定义

离散函数是有限整数区间或有限集合上的映射。对 $f:\{a, \dots, b+1\} \rightarrow \mathbb{R}$ ，前向差分定义为：

$$\Delta f(x) = f(x+1) - f(x)。$$

有限求和定义为：

$$\sum_{x=a}^b f(x) = f(a) + f(a+1) + \dots + f(b)。$$

这里的“积分”只是有限求和，不涉及极限。若步长为 h ，可定义 $\Delta_h f(x) = f(x+h) - f(x)$ ；若要近似连续导数，可另行保存缩放因子 $1/h$ 和误差界。

论证过程

离散微积分基本恒等式为：

$$\sum_{x=a}^b \Delta f(x) = f(b+1) - f(a)。$$

证明是望远镜相消：

$$(f(a+1) - f(a)) + (f(a+2) - f(a+1)) + \dots + (f(b+1) - f(b)) = f(b+1) - f(a)。$$

整个证明只包含有限项重排和加减法，因此适合整数、有理数、有限域或模环。若在模环中工作，等式按模数成立；若在机器字长中工作，等式按回绕语义成立。

实现要点

离散分析用于数值程序时，应把“定义层”和“近似层”分开。定义层中的差分、卷积、有限和都是精确离散对象；近似层才讨论它们与连续导数、积分或微分方程的关系。这样写可以避免把浮点误差误认为数学误差，也便于把同一算法迁移到整数硬件。

第4部分：离散几何

定义

离散几何的基本对象是有限网格、有限图或有限单纯复形。网格点可以写为整数向量 $p = (p_1, \dots, p_d) \in \mathbb{Z}^d$ 。常用距离包括：

$d_1(p, q) = \sum_i |p_i - q_i|$ ，即曼哈顿距离； $d_H(x, y) = \text{popcount}(x \oplus y)$ ，即汉明距离；图距离 $d_G(u, v)$ ，即最短路径边数。

离散线是满足相邻关系的有限点列。离散圆可以定义为 $\{p: d(p, c) = r\}$ 。离散面积或体积是区域内单元、格点或单形的计数，具体取决于建模约定。

论证过程

曼哈顿距离满足度量公理。非负性和对称性显然。三角不等式由整数绝对值不等式得到：

$$|p_i - r_i| \leq |p_i - q_i| + |q_i - r_i|。$$

对所有维度求和即可得 $d_1(p, r) \leq d_1(p, q) + d_1(q, r)$ 。图距离的三角不等式也直接来自路径拼接：从 u 到 v 的最短路和从 v 到 w 的最短路拼起来是一条从 u 到 w 的路，因此最短距离不会超过这条拼接路径长度。

实现要点

在 DCA 中，不宜把欧氏无理长度作为默认几何量。若需要欧氏几何，可使用整数平方距离 $\|p - q\|^2$ ，或者使用有理数近似并标明误差。对路径规划、图像处理、网格计算和芯片布线，曼哈顿距离、切比雪夫距离和图距离往往更贴近实现。

第5部分：逻辑与推理

定义

命题的真值取 $B=\{0,1\}$ 。基本逻辑运算定义为：

$\text{NOT } p = 1 \text{ xor } p$, $\text{AND}(p,q)=p \& q$, $\text{OR}(p,q)=p | q$, $p \rightarrow q = \text{OR}(\text{NOT } p, q)$ 。

含有 n 个布尔变量的命题公式可看作函数 $B^n \rightarrow B$ 。若对所有输入公式值都为 1，则称为重言式；若至少有一个输入为 1，则称为可满足。

论证过程

有限命题逻辑的判定过程非常直接：枚举 2^n

个赋值并计算公式值。正确性来自定义本身，因为每个可能世界都被枚举了一次。若公式经由合取范式表示，则 SAT 求解器可以通过 DPLL 或 CDCL

类算法避免全量枚举，但这些算法仍然保持有限搜索树。每次传播、分裂和回溯都在有限变量集上操作。

推理规则也可通过重言式验证。例如假言推理对应公式 $((p \rightarrow q) \text{ AND } p) \rightarrow q$ 。枚举 $p, q \in B$ 的四种情况，公式恒为 1，因此规则有效。

实现要点

把证明转化为布尔公式是 DCA 的重要工作流。小规模命题可直接真值表验证；大规模问题可交给 SAT/SMT 求解器；程序性质可先化为有限状态或有界模型检查问题。需要注意的是，“可判定片段”不等于所有数学命题都可自动判定，边界必须写清楚。

第6部分：离散傅里叶分析与数论变换

定义

对长度为 n 的有限序列 $a=(a_0, \dots, a_{n-1})$ ，若在有限域 F_p 中存在 n 次本原单位根 ω ，数论变换 NTT 定义为：

$$A_k = \sum_{j=0}^{n-1} a_j \omega^{jk} \bmod p。$$

逆变换为：

$$a_j = n^{-1} \sum_{k=0}^{n-1} A_k \omega^{-jk} \bmod p。$$

存在条件通常要求 $n \mid (p-1)$ ，并且 n 在 F_p 中可逆。常用素数如 998244353 是因为它支持很长的二的幂长度变换。

论证过程

逆变换正确性的核心是有限域中的正交关系：

$$\sum_{k=0}^{n-1} \omega^{k(j-l)} = n \text{ 当 } j=l, \text{ 否则为 } 0。$$

若 $j \neq l$ ，令 $r = \omega^{l-j}$ ，则 $r^n = 1$ 且 $r \neq 1$ 。有限等比和满足 $(r-1)\sum_{k=0}^{n-1} r^k = r^n - 1 = 0$ ，因为 $r-1$ 可逆，所以和为 0。代入逆变换公式后，只剩 a_j 项，故逆变换成立。

卷积定理也由有限求和换序得到：序列的循环卷积在 NTT 域中变成逐点乘法。这里没有浮点误差，只有模数选择、长度条件和溢出策略。

实现要点

NTT 是整数卷积、同态加密、多项式乘法和后量子密码中的常用工具。开源实现需写明素数、原根、最大长度、位逆序策略和负数归一化方式。若使用 CRT 合并多个模数，还应说明最终整数范围和重构条件。

第7部分：离散概率论

定义

有限概率空间由样本空间 $\Omega = \{\omega_1, \dots, \omega_N\}$ 和概率质量函数 $P: \Omega \rightarrow \mathbb{Q}_{\geq 0}$ 组成，满足 $\sum_{\omega} P(\omega) = 1$ 。若采用计数概率，则 $P(E) = |E|/|\Omega|$ 。

随机变量是函数 $X: \Omega \rightarrow S$ 。期望定义为有限和：

$$E[X] = \sum_{\omega \in \Omega} X(\omega) P(\omega)。$$

条件概率在 $P(B) > 0$ 时定义为：

$$P(A|B) = P(A \cap B) / P(B)。$$

论证过程

贝叶斯公式来自乘法交换：

$$P(A|B)P(B) = P(A \cap B) = P(B|A)P(A)。$$

只要分母不为 0，即可得 $P(A|B) = P(B|A)P(A)/P(B)$ 。若所有概率保存为整数权重，例如把概率写成同一分母下的有理数，整个推导只涉及整数加法、乘法和约分。

离散马尔可夫链由有限状态集和转移矩阵给出。 t 步分布是初始行向量右乘 P^t 。若状态和概率都用有理数表示，则矩阵幂仍然是有限代数运算。

实现要点

程序中尽量使用分数或整数权重，而不是默认浮点概率。对大规模模型可以用定点数近似，但必须保存缩放因子。概率论中的极限定理可作为外部分析工具；实现层常用有限样本偏差界、置信区间或组合不等式。

第8部分：离散微分几何

定义

离散微分几何把曲线、曲面和流形表示为有限网格、三角网格或单纯复形。一条离散曲线是点列 $(p_0, \dots, p_m)$ 。曲率可用转向角、方向变化或局部组合量表示。对三角网格，常用顶点曲率是角亏：

$$K(v) = 2\pi - \sum_{f \ni v} \theta_f(v)。$$

若希望完全整数化，可在规则三角网格中使用组合曲率，如 $K_c(v) = 6 - \deg(v)$ ，其中 $\deg(v)$ 是顶点度数。

论证过程

组合曲率的合理性来自局部平坦模型。规则三角网格中，一个内部平坦顶点通常有 6 个三角形围绕，故 $\deg(v)=6$ 时曲率为 0；少于 6 表示正曲率，多于 6 表示负曲率。对封闭三角剖分，欧拉示性数

$$\chi = V - E + F$$

是整数拓扑不变量。离散 Gauss-Bonnet 型结论把局部曲率和全局拓扑联系起来。若采用角亏公式，总曲率为 $2\pi\chi$ ；若采用组合归一化曲率，也可得到等价的整数恒等式。

实现要点

几何处理程序应记录网格的顶点表、边表、面表和邻接关系。曲率、法向、测地距离等连续概念可使用离散算子近似，但必须说明网格质量和误差来源。离散外微分给出了更系统的做法：把微分形式放在单形上，把外微分变成边界矩阵的转置。

第9部分：离散动力系统与整数 AI 运算

定义

离散动力系统是状态递推：

$$x_{t+1} = F(x_t)，$$

其中状态空间 S 是有限集合或有限编码集合， $F: S \rightarrow S$ 可计算。若 S 有限，则任意轨道必然最终进入循环。

整数 AI 运算把神经网络层写成整数或定点数函数。例如线性层可写为 $y = Wx + b$ ，其中 W, x, b 都是整数；激活函数可用 $\text{ReLU}(x) = \max(0, x)$ ；归一化和 softmax 可用查表、分段线性、定点指数或排序近似替代。

论证过程

有限动力系统的循环性由鸽巢原理证明。轨道 $x_0, x_1, \dots$ 中若前 $|S|+1$ 个状态都不同，就会超过状态空间大小，矛盾。因此存在 $i < j$ 使 $x_i = x_j$ 。由于递推函数确定，之后的序列以周期 $j-i$ 重复。

整数神经网络的正确性不是说它等同于实数网络，而是说在给量化规则下，它精确实现了一个离散函数。若量化映射为 $q(x) = \text{round}(x/s) + z$ ，则需要证明每层整数公式与量化规格一致，并给出误差界。

实现要点

AI 场景中应区分训练、推理和验证。推理可以完全整数化；训练通常仍依赖近似梯度，但可用定点梯度或量化感知训练降低差异。开源时建议把张量范围、缩放因子、溢出处理和舍入规则写入模型元数据。

第10部分：离散复分析与二元数

定义

若要在有限结构中模拟复数，可使用有限域扩张。对素数 p ，若 -1 在 F_p 中不是平方，可构造 $F_p[i]=F_p[x]/(x^2+1)$ ，元素写作 $a+bi$ 。若 $p\equiv 1\pmod 4$ ，则 i 已经在 F_p 中存在。

二元数或对偶数定义为 $a+b\varepsilon$ ，满足 $\varepsilon^2=0$ 。在离散实现中， a,b 可以取整数、模整数或有理数。若函数由加乘组成，则：

$$f(a+b\varepsilon)=f(a)+b f'(a) \varepsilon$$

对应前向模式自动微分。

论证过程

对偶数微分规则可由代数展开证明。以乘法为例：

$$(a+b\varepsilon)(c+d\varepsilon)=ac+(ad+bc)\varepsilon+b d\varepsilon^2=ac+(ad+bc)\varepsilon。$$

这正对应乘积求导法则

$(fg)'=f'g+fg'$ 。对多项式和由基本算术组合成的程序，可用结构归纳证明对偶数传播得到的 ε 系数等于导数或差分的线性化结果。

有限域复数的乘法封闭性来自商环定义：任何多项式都可模 x^2+1 化为一次式 $a+bi$ 。因此加乘都能落回同一有限集合。

实现要点

本文不把复分析中的全纯性、留数定理等连续结论直接搬到有限域中。更稳妥的做法是把“离散复分析”理解为有限域扩张、多项式映射和网格上的差分关系。二元数则更适合服务于自动微分和程序变换。

第11部分：离散微分方程

定义

离散微分方程在本文中指差分方程或递推系统。线性常系数差分方程可写为：

$$x_{t+k}=a_{k-1}x_{t+k-1}+\dots+a_0x_t+u_t。$$

把状态定义为 $s_t=(x_t,x_{t+1},\dots,x_{t+k-1})$ ，即可化为一阶系统：

$$s_{t+1} = A s_t + b u_t$$

若变量取有限环或有限域，则系统是有限状态机；若取有界整数，则仍可在有限窗口内精确计算。

论证过程

高阶递推化为一阶递推的正确性来自状态展开。 s_t 保存了计算下一项所需的全部历史值；矩阵 A 的前几行负责移位，最后一行负责代入递推公式。由此可以归纳证明：对任意 t ，矩阵系统产生的第一分量与原差分方程产生的 x_t 相同。

若状态空间有限，解轨道最终循环；若矩阵在有限域上可逆，则状态转移是置换，所有轨道从一开始就在某个循环中。

实现要点

工程上应优先写清楚采样周期、状态向量、输入边界和数值范围。很多连续微分方程的求解器最终都落到差分方程。DCA 视角不是否认连续模型，而是要求把实际执行的离散递推写成独立、可验证的对象。

第12部分：离散优化与控制

定义

离散优化问题可写为：

minimize $f(x)$ ，其中 $x \in X$ ， X 是有限集合。

若有约束，则写为 $g_i(x) \leq 0$ 或布尔谓词 $C_i(x)=1$ 。离散控制问题通常包含有限状态集 S 、动作集 A 、转移函数 $T:S \times A \rightarrow S$ 和代价函数 $c:S \times A \rightarrow \mathbb{Z}$ 。

有限时域动态规划的值函数定义为：

$$V_t(s) = \min_{a \in A(s)} [c(s,a) + V_{t+1}(T(s,a))],$$

终端条件为 $V_T(s) = h(s)$ 。

论证过程

动态规划正确性来自最优子结构。任意从时刻 t 开始的策略，第一步选择某个动作 a ，之后剩余部分是一条从 $T(s,a)$ 到终点的策略。若后续策略不是该子问题最优，就能替换为更优策略，从而改进整体，矛盾。因此最优值满足 Bellman 递推。

由于 S 、 A 和时间 T 都有限，值迭代需要的计算步数有限。若成本为整数，整个过程只含整数加法、比较和取最小值。

实现要点

离散优化不一定需要梯度。枚举、分支定界、动态规划、整数规划、SAT/SMT 编码都属于自然工具。控制问题中若使用连续状态，应说明离散化网格和误差；若状态本来就是程序状态、库存、任务队列或图节点，则可以直接使用 DCA

形式。

第13部分：离散信息论与编码

定义

信息源是有限字母表 Σ 上的符号序列生成机制。若符号 x 的概率为 p_x ，Shannon 熵为：

$$H(X) = -\sum_x p_x \log(1/p_x)。$$

实现层中可把 p_x 保存为整数计数，把对数表保存为定点数。编码是映射 $C: \Sigma \rightarrow \{0,1\}^*$ 。若任意码字都不是另一个码字的前缀，则称为前缀码。

纠错码是集合 $C \subset \Sigma^n$ 。最小距离 $d_{\min}$ 是不同码字之间的最小汉明距离。若 $d_{\min} \geq 2t+1$ ，则可纠正至多 t 个错误。

论证过程

纠错能力证明很直接。若接收字 y 距离真实码字 c 至多 t ，同时也距离另一码字 $c' \neq c$ 至多 t ，则由三角不等式：

$$d(c, c') \leq d(c, y) + d(y, c') \leq 2t。$$

这与 $d_{\min} \geq 2t+1$ 矛盾。因此半径 t 的球互不相交，最近邻译码唯一。

前缀码可即时解码，因为读入比特流时，一旦匹配到某个码字，就不可能还只是更长码字的前缀。有限树遍历即可完成解码。

实现要点

编码理论非常适合 DCA：字母、码字、校验矩阵、综合和译码表都是有限对象。需要实数的地方通常是性能分析，如容量和熵；实现层可使用整数计数、定点对数和有限块长界。

第14部分：从数学定义到指令集

定义

指令集是数学运算的机器接口。一个简化的 DCA-ISA 至少包含：位逻辑 AND/OR/XOR/NOT，整数算术 ADD/SUB/MUL/DIV，移位 SHL/SHR/SAR，比较 CMP，内存访问 LOAD/STORE，控制流 JMP/BRANCH。

若要支持前文算法，可扩展：

MAJ 用于全加器进位；MADD 用于矩阵乘和卷积；BITREV 用于 NTT；POPCNT 用于汉明距离；MIN/MAX 用于优化和 ReLU。

论证过程

从数学定义到指令集的关键是语义保持。以加法为例，指令 $\text{ADD}_w(x,y)$ 的规格是返回 $(x+y) \bmod 2^w$ 。硬件电路若由全加器串联组成，则第1部分的位归纳证明同时证明了该指令实现规格。

对复合程序，可用组合性证明：若每条指令满足局部规格，且寄存器、内存和程序计数器的状态转移定义清楚，则程序执行轨迹满足由这些局部规格组合出的全局规格。

实现要点

开源时不必真的设计芯片，但应把“操作语义”写清楚。RISC-V 这类开放指令集的文档方式值得参考：每条指令给出编码、输入、输出、异常和边界行为。DCA 文档也应如此，避免“数学上加法”和“机器上加法”混用。

第15部分：离散拓扑与组合同调

定义

单纯复形由顶点、边、三角形及更高维单形组成，并满足：若一个单形属于复形，则它的所有面也属于复形。 k 链是 k 维单形的整数或模 p 系数形式和。边界算子 $\partial_k: C_k \rightarrow C_{k-1}$ 把单形映到其有符号边界。

同调群定义为：

$$H_k = \ker(\partial_k) / \text{im}(\partial_{k+1}).$$

直观上， $\ker(\partial_k)$ 是没有边界的 k 维循环， $\text{im}(\partial_{k+1})$ 是高一维对象的边界。

论证过程

同调成立的核心恒等式是 $\partial_k \partial_{k+1} = 0$ ，即“边界的边界为空”。以三角形 $[a,b,c]$ 为例：

$$\partial[a,b,c] = [b,c] - [a,c] + [a,b].$$

再取边界，所有顶点项两两相消。因此任何高维边界都是循环， $\text{im}(\partial_{k+1}) \subset \ker(\partial_k)$ ，商结构定义良好。

Betti 数可由边界矩阵的秩计算：

$$\beta_k = \dim \ker(\partial_k) - \dim \text{im}(\partial_{k+1}).$$

在有限域上，这就是有限矩阵高斯消元。

实现要点

拓扑数据分析、网格连通性、孔洞检测都可写成有限线性代数问题。实现时需要统一顶点排序和单形方向，否则边界矩阵符号会错。若只关心模 2 同调，可省略方向符号，计算更简单。

第16部分：有限域上的代数几何

定义

在 DCA 中，代数几何可先限制在有限域 F_q 上。给定多项式集合 $f_1, \dots, f_m \in F_q[x_1, \dots, x_n]$ ，其有限域解集为：

$$V(F_q) = \{x \in F_q^n : f_i(x) = 0 \text{ for all } i\}.$$

这是一个有限集合。多项式求值、加法、乘法、约化都在有限域中进行。射影空间、曲线和码也可以用有限域坐标表示。

论证过程

解集有限性显然： F_q^n 只有 q^n 个点。判定某点是否在簇上，只需对每个多项式求值。若直接枚举，最多 $q^n m$ 次求值；若规模较大，可用 Groebner 基、消元、线性代数或专门的有限域算法减少计算。

代数几何码的离散性也来自有限域。选取有限域曲线上的有理点，把函数在这些点的取值组成码字。所有取值都在有限域中，因此编码和校验矩阵都可精确存储。

实现要点

有限域代数几何不需要浮点，但需要非常谨慎的域实现。应记录域的阶、不可约多项式、元素表示、乘法表或乘法算法。对密码和编码应用，还要区分数学正确性和安全性假设，后者不能只靠代数定义证明。

第17部分：构造数学与类型论

定义

构造数学强调：证明一个对象存在，通常要给出构造它的方法。类型论把命题看作类型，把证明看作该类型的项。归纳类型由有限构造子生成，例如自然数：

```
Inductive nat :=  
| 0 : nat  
| S : nat -> nat.
```

有限列表、有限树、语法树、证明树都可用归纳类型表示。递归函数必须按结构下降或提供终止性论证。

论证过程

结构归纳是归纳类型的基本证明原则。若要证明性质 $P(n)$ 对所有自然数成立，只需证明 $P(0)$ ，并证明 $P(n) \rightarrow P(Sn)$ 。这是因为自然数项只能由有限次 S 作用在 0 上构造出来。

程序终止性也可用类似方式证明。若递归调用总是在更小的结构上进行，例如列表尾部或子树，则不可能无限下降，因此函数有限步结束。

实现要点

证明助手适合表达 DCA 的核心约定：比特串、字长、有限向量、矩阵和边界矩阵都能作为类型。相比纸面证明，形式化版本会强迫作者写清楚每个隐含前提，例如维度相等、模数非零、分母可逆等。

第18部分：离散自动微分

定义

自动微分把程序看作由基本运算组成的计算图。前向模式可用对偶数 $a+b\epsilon$ ；反向模式保存 Wengert tape，即每个中间变量及其依赖关系。

对整数或定点程序，梯度不一定是实数意义下的导数。本文可采用三种替代：

1. 对多项式整数程序使用形式导数；
2. 对差分程序使用前向差分或有限差分；
3. 对量化神经网络使用直通估计器或离散替代梯度，并明确它是训练启发式。

论证过程

反向模式的正确性来自链式法则的拓扑逆序应用。计算图是有限有向无环图。每个节点 v 的值由父节点计算得到，目标为 L 。反向传播维护伴随量 $\text{adj}[v] = \partial L / \partial v$ 。从输出开始逆序处理时，若 u 是 v 的父节点，则累加：

$$\text{adj}[u] += \text{adj}[v] * \partial v / \partial u。$$

由于所有从 u 到输出的路径都会在逆序处理中被考虑一次，最终 $\text{adj}[u]$ 等于所有路径贡献之和。

实现要点

整数自动微分需要明确舍入和不可导点。ReLU 在 0 处、取整函数、查表函数都不是普通光滑函数。开源实现应把训练用梯度和推理用整数函数分开描述，避免把训练近似误写成推理语义。

第19部分：从定义到形式化验证

定义

形式化验证把对象、程序和定理写入证明系统或可检查逻辑。DCA 中常见对象包括：

bit, word w , 有限向量 $\text{Vec } A^n$, 矩阵 $\text{Mat } R^{m \times n}$, 图 $\text{Graph } n$, 以及程序状态 State 。

规格是谓词或关系，例如：

$$\text{add_spec}(a,b,r) := \text{val}(r) = (\text{val}(a) + \text{val}(b)) \bmod 2^w。$$

实现正确性是定理：

$$\forall a \ b, \text{add_impl}(a,b) \text{ satisfies } \text{add_spec}。$$

论证过程

验证链条通常分三层。第一层证明基础运算正确，如全加器、乘法器、比较器。第二层证明算法正确，如矩阵乘法、NTT、动态规划。第三层证明系统组合正确，如编译器保持语义、指令实现满足 ISA 规格。

每层都依赖组合性：若组件规格成立，且组件连接方式满足接口条件，则整体规格成立。形式化工具的价值在于检查接口条件是否遗漏。

实现要点

本文不要求所有内容都立刻形式化，但写作风格应便于形式化。每个定义最好说明载体、运算、前提和输出。证明不要只写“显然”，而应指出使用的是有限枚举、归纳、矩阵代数还是循环不变式。

第20部分：离散随机过程与鞅

定义

离散时间随机过程是随机变量序列 $X_0, \dots, X_T$ ，定义在有限概率空间上。过滤 F_t 表示到时刻 t 可观察的信息。若对所有 t 有：

$$E[X_{t+1}|F_t] = X_t,$$

则称 X_t 为鞅。若条件期望大于等于或小于等于当前值，则分别是次鞅或上鞅。

论证过程

有限情形下，条件期望可以直接按等价类求和。给定 F_t 的一个信息块 C ，有：

$$E[X_{t+1}|C] = \sum_{\omega \in C} X_{t+1}(\omega) P(\omega) / P(C).$$

若每个信息块上该值等于 X_t ，鞅条件成立。停时定理在有界时间和有限状态下可通过反复使用条件期望证明：若 $\tau \leq T$ ，则把过程在停时后冻结，得到新过程 $Y_t = X_{\min(t, \tau)}$ 。若 X_t 是鞅，则 Y_t 仍是鞅，于是 $E[Y_T] = E[Y_0]$ ，即 $E[X_\tau] = E[X_0]$ 。

实现要点

金融二叉树、随机游走、排队系统和在线算法分析都可以用有限随机过程表达。概率建议保存为整数权重或分数。若使用伪随机数，应把生成器也纳入状态转移，而不是把随机性当成不可见外部魔法。

第21部分：离散度量空间

定义

离散度量空间是有限集合 X 与函数 $d: X \times X \rightarrow \mathbb{Q}_{\geq 0}$ ，满足：

1. $d(x, y) = 0$ 当且仅当 $x = y$ ；
2. $d(x, y) = d(y, x)$ ；

3. $d(x,z) \leq d(x,y) + d(y,z)$ 。

常用例子是汉明空间 $\{0,1\}^n$ 、带权图最短路空间、整数格上的 L1 距离。等距嵌入是映射 $\phi: X \rightarrow Y$ ，满足 $d_X(x,y) = d_Y(\phi(x), \phi(y))$ 。

论证过程

汉明距离的三角不等式可逐位证明。对任意一位，如果 $x_i \neq z_i$ ，则在二值集合中不可能同时有 $x_i = y_i$ 且 $y_i = z_i$ ，因此该位对 $d(x,z)$ 的贡献不超过它对 $d(x,y) + d(y,z)$ 的贡献。对所有位求和即可。

图最短路距离的证明同第4部分，来自路径拼接。嵌入问题的验证则是有限对检查：枚举所有 $x, y \in X$ ，比较距离是否保持。

实现要点

离散度量空间适合最近邻搜索、聚类、纠错码、图嵌入和表示学习。大规模实现不一定保存完整距离矩阵，可以保存生成规则或稀疏邻接表。若只近似保持距离，应给出失真界 α ，例如 $d_Y(\phi(x), \phi(y))$ 介于 d_X 和 αd_X 之间。

第22部分：有限维算子代数

定义

在 DCA 中，算子可先理解为有限矩阵。给定环或域 R ， $n \times n$ 矩阵集合 $\text{Mat}_n(R)$ 在矩阵加法和乘法下形成代数。矩阵的伴随、范数和谱概念若涉及实数或复数，需要离散替代。

可使用的整数范数包括：

$$\|A\|_1 = \max_j \sum_i |A_{ij}|, \quad \|A\|_\infty = \max_i \sum_j |A_{ij}|。$$

若在有限域上工作，则“大小”范数不再自然，更多关注秩、核、像、不变量因子和特征多项式。

论证过程

矩阵乘法的次乘性在 1 范数下可证明：

$$\|AB\|_1 = \max_j \sum_i |\sum_k A_{ik} B_{kj}|$$

$$\leq \max_j \sum_i \sum_k |A_{ik}| |B_{kj}|$$

$$= \max_j \sum_k |B_{kj}| \sum_i |A_{ik}|$$

$$\leq \|A\|_1 \|B\|_1。$$

这说明有限维算子可以用整数不等式分析稳定性。若采用有理数或定点数，证明仍可保留。

实现要点

不要轻易把无限维 C^* -代数结论照搬到有限矩阵。工程上更可靠的是写清楚矩阵维数、底层数域和所用范数。对量子信道、马尔可夫算子和图拉普拉斯，有限矩阵模型通常已经足够表达可计算部分。

第23部分：离散信号处理

定义

离散信号是有限序列 $x[0], \dots, x[N-1]$ 。有限冲激响应滤波器定义为卷积：

$$y[n] = \sum_{k=0}^{M-1} h[k]x[n-k]。$$

抽取 $\text{downsample}_M(x) = x[0], x[M], x[2M], \dots$ 。插值可先零插入，再滤波。小波变换可通过提升方案实现，其中预测、更新和舍入都可以是整数操作。

论证过程

卷积的线性性由有限求和分配律证明：

$$h^*(ax+bz) = a(h^*x) + b(h^*z)。$$

多采样率系统中的关键是索引变换。抽取和卷积一般不交换，必须根据 $y[m] = \sum_k h[k]x[mM-k]$ 重新推导。整数小波提升的可逆性来自每一步都是可逆整数变换。例如：

$$d = b - a, \quad s = a + \text{floor}(d/2)。$$

给定 s, d ，可恢复 $a = s - \text{floor}(d/2)$ ， $b = a + d$ 。

实现要点

信号处理中的正弦、频率响应和谱图常用于分析，但实际音视频编码、图像处理和通信系统大量使用整数滤波、定点 FFT/NTT 和查表。文档应区分分析用浮点图示和实现用整数管线。

第24部分：DCA-ISA 与微架构草图

定义

DCA-ISA 是面向前述有限运算的指令接口草图，不是必须新造硬件。它可以视为一个“最小整数计算目标”。状态包括寄存器文件、内存、程序计数器和标志位。每条指令是状态转移函数：

$$\text{step: State} \times \text{Instr} \rightarrow \text{State}。$$

微架构则是实现该状态转移的电路组织，例如 ALU、乘法器、除法器、加载存储单元、矩阵乘协处理器、NTT 协处理器和控制单元。

论证过程

ISA 正确性与微架构正确性分开证明。ISA

层定义抽象状态转移；微架构层证明若流水线、旁路、缓存和异常处理完成一次指令提交，则提交效果等于 ISA 的 step。这种证明通常用不变式：流水线中每条未提交指令的操作数、目的寄存器和顺序关系都与抽象执行前缀一致。

对于协处理器，如 NTT 单元，只需证明其输入输出满足第6部分的变换规格，不要求上层知道内部蝶形调度。

实现要点

实际可从软件模拟器开始，而不是硬件。先写解释器和测试，再把热点运算映射到 SIMD、GPU、FPGA 或 ASIC。只要接口和规格稳定，底层实现可以逐步替换。

第25部分：操作系统与认证内核

定义

操作系统内核可以看成有限状态机。状态包括任务表、页表、调度队列、文件描述符表、权限位图和设备状态。系统调用是受控状态转移：

$\text{syscall}: \text{KernelState} \times \text{UserRequest} \rightarrow \text{KernelState} \times \text{Response}.$

认证内核要求为关键性质写出规格，如内存隔离、权限检查、调度公平性或中断处理不破坏内核不变量。

论证过程

内存隔离的典型证明是不可达性不变式。对任意用户进程 p ，它可访问的地址集合由页表和权限位定义。若每个内存访问指令都先检查地址属于该集合，且系统调用修改页表时保持不同进程授权区域不非法重叠，则归纳可得：执行任意有限步后，进程 p 仍不能读写未授权页。

调度器可用队列不变式证明：就绪任务要么在队列中，要么正在运行；每次切换保持该不变式。

实现要点

OS 里的“数学”不是抽象口号，而是位图、整数边界、访问控制矩阵和状态机。DCA 写法适合小内核、嵌入式系统和安全关键组件。范围越小，形式化越可行。

第26部分：有限域量子模型

定义

这一部分只讨论有限域上的量子式线性模型，不声称替代物理量子力学。设状态是有限域 F_q 上的向量

$\psi \in F_q^{n_0}$ 。门是可逆线性变换

$U \in GL_n(F_q)$ 。测量可定义为把状态映射到有限结果集合的规则，例如按某个函数或离散权重归一化后给出结果。

若要模拟量子线路结构，可保留张量积、可逆门、受控门和 oracle 查询等组合关系，但振幅不再是复数概率幅。

论证过程

可逆门保持信息不丢失：若 U 可逆，则从 $\psi \otimes \psi = U\psi$ 可唯一恢复 $\psi = U^{-1}\psi \otimes \psi$ 。多个可逆门组合仍可逆，因为 $(UV)^{-1} = V^{-1}U^{-1}$ 。因此有限域线路的演化是有限集合上的置换或可逆线性变换。

若使用 Deutsch-Jozsa

这类算法的离散类比，必须重新证明干涉或区分性质在所选有限域模型下成立，不能直接引用复 Hilbert 空间结论。

实现要点

有限域量子模型适合研究可逆计算、线性代数玩具模型和量子算法的结构骨架。真正的量子概率、测量公理和物理实现需要复数 Hilbert 空间。本文只保留可有限编码的部分，并把物理解释放在边界之外。

第27部分：离散时空与因果元胞自动机

定义

离散时空模型把空间设为整数格点，把时间设为整数步。事件写为 (t, x) 。局部因果律可定义为：

$\text{state}_{t+1}(x)$ 只依赖 $\text{state}_t(y)$ ，其中 $d(x, y) \leq r$ 。

这里 r 是传播半径。若 $r=1$ ，信息每步最多传播一格。元胞自动机是局部规则：

$F: \text{local_neighborhood} \rightarrow \text{cell_state}$ 。

论证过程

因果锥性质可用归纳证明。 $t=0$ 时，某点只受自身影响。若经过 k 步，点 x 只受距离不超过 kr 的初始点影响，则第 $k+1$ 步时， x 的邻域点各自只受距离不超过 kr 的点影响，合并后不超过 $(k+1)r$ 。因此有限速度传播成立。

离散波动、扩散或场方程若写成局部更新规则，也可用同样方法证明依赖域有限。

实现要点

相对论中的连续洛伦兹变换不能简单整数化。DCA 版本更适合表达“局部性、有限传播速度、守恒量和可模拟性”。若要近似连续物理，应给出网格尺度、稳定性条件和误差分析。

第28部分：计算复杂性

定义

计算问题是语言 $L \subseteq \{0,1\}^*$ 或函数 $f: \{0,1\}^* \rightarrow \{0,1\}^*$ 。算法的时间复杂度是输入长度 n 到步数上界的函数 $T(n)$ ，空间复杂度类似。

类 P 包含存在多项式时间算法的问题。类 NP 包含存在多项式长度证书且可多项式时间验证的问题。 NP 完全性通常通过多项式时间归约定义。

论证过程

SAT 属于 NP，因为给定变量赋值，可在公式长度的多项式时间内检查每个子句是否满足。若一个问题 A 可多项式时间归约到 B，且 B 可多项式时间求解，则 A 也可多项式时间求解：先把 A 的输入变换为 B 的输入，求解 B，再把答案解释回 A。

渐近记号并不要求一次性处理无限对象。它表达的是：存在一个可计算界函数，使任意给定输入长度都有有限资源上界。

实现要点

复杂性分析是对算法族的说明，不是对某次运行的测量。DCA 写法中应同时给出理论上界和具体实现参数。对固定小规模问题，指数算法也可能可接受；对输入增长的问题，资源界必须明确。

第29部分：元胞自动机与计算普适性

定义

元胞自动机由格点集合、有限状态集合、邻域和局部更新规则组成。对一维半径 r 自动机：

$$x_{\{t+1\}}(i)=F(x_t(i-r),\dots,x_t(i+r))。$$

对二维生命游戏，状态为活/死，下一步由周围 8 个邻居的活细胞数量决定。Rule 110 是一维二值元胞自动机，其局部规则可由 8 种三元邻域的输出表给出。

论证过程

元胞自动机的全局更新是有限或可生成配置上的确定函数。若工作在有限环面网格上，状态数有限，轨道必然最终循环。若允许无限但有限支撑的配置，则每次有限步只影响有限因果锥内的区域。

计算普适性证明通常不是穷举，而是构造模拟：证明某个自动机可实现逻辑门、存储、线路同步或标签系统，再引用这些模型与图灵机等价。Rule 110 的普适性就是此类构造性结果。

实现要点

元胞自动机很适合展示“简单局部规则产生复杂全局行为”。但开源文档应避免把复杂行为直接解释成智能或物理定律。可靠说法是：它提供了有限状态、局部更新和可计算模拟的实验平台。

第30部分：形式化验证闭环

定义

验证闭环指从规格、实现、证明到测试的循环。一个最小闭环包括：

1. 数学规格 Spec；

2. 程序实现 Impl;
3. 定理 Impl refines Spec;
4. 可执行测试用例;
5. 版本化的证明脚本或检查日志。

编译器验证还需要语义保持：源程序语义与目标程序语义一致。

论证过程

闭环的核心是精化关系。若抽象状态 A 与具体状态 C 之间有关系 $R(A,C)$ ，并且每个具体步都能对应一个抽象步或允许的內部步，则具体实现精化抽象规格。归纳地，初始状态满足 R，每步保持 R，所以任意有限执行前缀都满足规格。

矩阵乘法、排序、解析器、内存分配器都可用类似方法写证明：定义抽象规格，给出循环不变式，证明每次循环推进保持不变式并最终满足后置条件。

实现要点

不要把测试当成证明，也不要把证明当成性能测试。二者互补：测试帮助发现规格误解和工程错误，证明覆盖理论状态空间。DCA 文档建议每个核心算法同时给出小例子、性质测试和证明草图。

第31部分：离散信息几何

定义

有限概率分布可表示为整数权重向量 $p=(p_1,...,p_n)$ ，总权重 $M=\sum p_i$ 。归一化概率是 p_i/M 。两个分布的总变差距离可写为：

$$TV(p,q)=1/2 \sum_i |p_i/M - q_i/M|。$$

若 p,q 总权重相同，则整数形式为：

$$TV_M(p,q)=1/2 \sum_i |p_i-q_i|。$$

离散 Fisher 信息可用参数差分替代导数，例如 $\Delta_\theta \log p_\theta(x)$ ，再用有限和求期望。

论证过程

总变差距离满足度量性质。非负性和对称性显然；若距离为 0，则每项绝对值为 0，故 $p=q$ 。三角不等式来自绝对值三角不等式：

$$|p_i-r_i| \leq |p_i-q_i| + |q_i-r_i|。$$

求和并除以 2 即得。由于所有量都可用整数权重保存，总变差适合作为离散概率模型之间的基本距离。

实现要点

连续信息几何依赖光滑流形和微分结构。DCA 中更适合使用有限分布族、整数权重、差分和组合散度。若使用 KL 散度，需要对数；可用有理上下界、定点表或只在分析层使用。

第32部分：符号动力学与离散混沌

定义

符号动力学研究有限字母表上的序列及移位映射。给定字母表 Σ ，双边序列空间中的移位为：

$$(\sigma x)_t = x_{t+1}。$$

有限型子移位由一组禁止词定义：所有不包含禁止词的序列构成系统。若只实现有限窗口，则状态可表示为长度 k 的词，转移由允许拼接关系给出。

离散混沌映射可用模整数映射表示，例如 Arnold cat map 的模 N 版本。

论证过程

有限型子移位可转化为有限图。图的顶点是长度 $k-1$ 的允许词，边表示可拼接成长度 k 的允许词。于是合法序列对应图上的路径。长度 n 的合法词数量可由邻接矩阵幂计算，这把符号动力学问题转化为有限线性代数。

模 N 的 cat map 若矩阵行列式与 N 互素，则矩阵在 $\mathbb{Z}/N\mathbb{Z}$ 上可逆，因此映射是有限集合上的置换。所有轨道最终周期化，周期可计算。

实现要点

“混沌”在有限机器上通常表现为长周期、敏感依赖和统计复杂性，而不是真正无限精度意义下的混沌。文档应写明状态空间大小和周期检测方法。

第33部分：离散最优控制

定义

离散最优控制研究有限时间或无限折扣时间上的状态动作系统。有限时间问题为：

$$s_{t+1} = T(s_t, a_t), \quad J = \sum_{t=0}^{T-1} c(s_t, a_t) + h(s_T)。$$

目标是选择策略 $\pi_t: S \rightarrow A$ 使 J 最小。若状态和动作有限，可直接用动态规划；若状态是整数格，可在有界区域内近似。

论证过程

Hamilton-Jacobi-Bellman 方程在离散情形就是 Bellman 递推。正确性与第12部分类似，但这里强调策略。定义 $\pi_t^*(s)$ 为使 Bellman 右侧取最小的动作。对时间反向归纳可证明：从任意 t,s 出发，执行 π^* 得到的代价等于 $V_t(s)$ ，任何其他策略代价不小于它。

若使用折扣无限时域，有限状态和折扣因子 $\gamma < 1$ 可保证 Bellman 算子是压缩映射；实现中通常用定点或有理数近似迭代。

实现要点

机器人导航、游戏 AI、库存控制和任务调度经常本来就是离散控制问题。连续 LQR 等方法可作为背景，但 DCA 文档应优先给出状态编码、动作集合、代价整数化和边界条件。

第34部分：代数编码与密码学

定义

代数编码使用有限域上的线性空间。线性码 C 是 F_q^n 的子空间，可由生成矩阵 G 或校验矩阵 H 表示：

$C = \{mG : m \in F_q^k\}$ ， $c \in C$ 当且仅当 $Hc^T = 0$ 。

密码学中，有限域、椭圆曲线、格、多项式环和哈希函数都是有限可计算对象。后量子格密码常在模整数多项式环上操作。

论证过程

线性码的 syndrome 译码基于等式 $H(c+e)^T = Hc^T + He^T = He^T$ 。由于码字项为 0，综合只依赖错误向量。若预先建立低重量错误到综合的查表，即可纠错。

格密码的正确性通常证明为：加密后解密得到 $m + \text{noise}$ ，其中噪声幅度小于取整阈值，所以舍入恢复 m 。安全性则依赖问题困难性假设，如 LWE 或 Module-LWE，不能由本文的 DCA 公理直接推出。

实现要点

密码实现必须区分功能正确、安全证明和抗侧信道实现。DCA 可以帮助写清有限环运算和边界，但不能替代密码分析。参考 NIST 标准时，应使用标准参数和测试向量。

第35部分：DCA 的表达范围

定义

DCA 的表达范围可以概括为：凡是能用有限编码表示、能用有限规则操作、能给出有限验证过程的对象，都适合纳入本文框架。包括有限组合数学、图论、有限代数、程序语义、有限概率、离散几何和可执行算法。

不适合直接纳入实现层的对象包括：不可构造的任意实数、无穷维空间的完整对象、依赖选择公理的非构造存在性、以及没有有限误差界的连续近似。

论证过程

这个范围不是本体论结论，而是工程边界。计算机在任意时刻只能保存有限位，任意程序运行也只能经历有限步。因此若目标是实现、验证或复现，有限编码是必要条件。反过来，只要对象有有限编码且操作有有限算法，就可以在某种机器模型上执行。

与皮亚诺算术的关系可以理解为：自然数和递归定义是核心基础，但 DCA 更强调具体编码、字长、资源边界和程序接口。

实现要点

写开源文档时，建议避免“某对象不存在”这类过强说法。更准确的表述是：“本文的实现层不直接表示该对象；若需要它，应通过有限近似、生成器、规格或证明接口进入系统。”

第36部分：物理实现蓝图

定义

物理实现蓝图是一台整数计算系统的分层草图：

硬件层：比特、寄存器、ALU、内存、互连；指令层：整数 ISA 与状态转移语义；系统层：内核、调度、内存隔离；库层：有限代数、矩阵、NTT、图算法；应用层：信号处理、优化、AI 推理、验证工具。

论证过程

分层正确性的论证依赖接口契约。硬件满足指令规格；指令规格支撑编译器目标语义；编译器保持源程序语义；库函数满足数学规格；应用程序调用库函数并保持自身不变式。若每层都有清楚的前置条件和后置条件，则可以组合为端到端正确性论证。

性能分析也可离散化：面积用门数或平方微米估计，功耗用整数计数近似，延迟用周期数，内存用字节数。它们都能进入整数优化模型。

实现要点

这只是工程蓝图，不必一次做完整硬件。更现实的路线是先做参考解释器、测试集、形式化片段和整数数学库，再逐步替换底层加速器。

第37部分：离散微分拓扑与 Morse 理论

定义

离散 Morse 理论在有限单纯复形或 CW 复形上工作。一个离散 Morse 函数给每个单形赋值，并限制每个单形附近“违反维度单调性”的邻居数量。未被配对的单形称为临界单形。

离散梯度向量场由单形和其一个上维共面的配对组成，要求不存在闭合梯度路径。临界单形数量与空间的同调信息相关。

论证过程

离散 Morse 理论的核心思想是：若一个低维单形与一个高维单形成对，且这种配对不形成闭环，则可以把它们作为可消去的局部结构。每次消去不改变同伦型。反复消去后，剩下的临界单形给出一个更小的复形模型。

Morse 不等式说明临界单形数 m_k 至少覆盖 Betti 数 β_k ，即 $m_k \geq \beta_k$ 的弱形式。直观上，同调中的每个独立孔洞都必须由某些临界结构承载。

实现要点

离散 Morse 理论非常适合网格简化、拓扑降噪和数据形状分析。实现重点是维护单形邻接、配对关系和无闭环条件。所有操作都是有限组合操作。

第38部分：离散谱理论

定义

给定有限图 $G=(V,E)$ ，邻接矩阵为 A ，度矩阵为 D ，图拉普拉斯为：

$L=D-A$ 。

L 是整数矩阵。谱理论通常研究特征值，但在 DCA 中可优先研究整数可计算的不变量：核维数、秩、连通分量数、二次型 $x^T L x$ 、割大小和边界算子。

论证过程

关键恒等式为：

$$x^T L x = \sum_{\{(u,v) \in E\}} (x_u - x_v)^2。$$

展开右侧，每条边贡献 $x_u^2 + x_v^2 - 2x_u x_v$ ，汇总后得到度数项减邻接项。由此可知 $x^T L x \geq 0$ ，且若它为 0，则每条边两端 $x_u = x_v$ 。因此 $Lx=0$ 的向量在每个连通分量上常数，核维数等于连通分量数。

实现要点

谱聚类常用实特征向量，但许多图问题可用整数替代方法：连通分量用 BFS/DFS，割问题用最大流/最小割，调和函数可解有理线性方程。若确实计算特征值，应说明浮点近似和误差。

第39部分：离散神经网络架构搜索

定义

神经网络架构搜索 NAS 可写成有限优化问题。定义操作集合 O ，层数上限 L ，通道选项、连接选项和量化位宽选项都取有限集合。一个架构是有限编码串：

arch=(op_1,ch_1,skip_1,...,op_L,ch_L,skip_L)。

评价函数包括准确率、延迟、参数量、内存和能耗。若全部测量或估计为整数，就得到多目标整数优化问题。

论证过程

搜索空间有限，因此最优架构存在。若使用穷举，正确性显然但代价高。若使用随机搜索、进化算法或贝叶斯优化，则它们是启发式，不能保证找到全局最优，除非附加枚举覆盖或分支定界证明。

资源约束可写成谓词 $R(\text{arch}) \leq B$ 。只要构造、变异和筛选步骤都保持或检查该谓词，就能保证输出架构满足预算。

实现要点

NAS 文档中应避免只报告“找到更好模型”。需要保存搜索空间定义、随机种子、评价协议、硬件目标和约束。DCA 写法能帮助复现：架构编码是有限串，评价过程是可执行程序。

第40部分：DCA 自举解释器

定义

自举解释器是用 DCA 可表达语言实现的 DCA 语言解释器。程序本身可表示为语法树或字节码列表。解释器状态包括环境、栈、堆和当前指令。

eval: Program $\times$ Input $\times$ Fuel $\rightarrow$ Result。

这里 Fuel 是步数上界，用来保证解释器函数本身总是终止。若 fuel 耗尽，则返回未完成状态。

论证过程

解释器正确性可用小步语义证明。先定义语言的抽象执行关系 $P, s \rightarrow s'$ ；再证明解释器每执行一个字节码步骤，都模拟一次抽象小步。对执行步数归纳可得：若抽象程序在 k 步后得到结果，且 $\text{Fuel} \geq k$ ，解释器也得到同一结果。

自举并不神秘：只要语言能表示自身语法，并能解释这些语法对象，就可以写自解释器。反射能力来自程序可作为数据被读取和变换。

实现要点

自举系统容易变复杂。建议先实现极小核心语言：整数、布尔、条件、循环、函数调用和数组。之后再加入类型检查、编译和优化。每个扩展都应有语义规则和测试。

第41部分：离散物理映射

定义

离散物理映射把物理模型的可计算部分写成有限状态或格点更新。典型对象包括格点场、粒子系统、守恒量、局部作用量和有限差分方程。离散作用可写为：

$$S[\text{path}] = \sum_t L(x_t, x_{t+1})。$$

离散变分问题是在有限路径集合中寻找使 S 取极小或驻定的路径。

论证过程

离散 Euler-Lagrange 方程可通过有限变量扰动推导。若路径内部点 x_t 改变，只影响相邻两项 $L(x_{t-1}, x_t)$ 与 $L(x_t, x_{t+1})$ 。令离散差分的一阶变化为 0，即得到局部方程。这一推导只涉及有限求和和有限差分。

离散 Noether 型结论也可写成：若局部更新规则在某个有限变换群下不变，则对应的组合量在更新中保持。具体守恒量需要逐个模型证明。

实现要点

物理映射最容易越界。本文只建议把离散模型作为计算模型或数值模型，不直接宣称它就是现实底层结构。若要与实验物理关联，必须给出尺度、误差和可检验预测。

第42部分：定理证明自动化

定义

自动定理证明把命题转化为可由程序搜索的形式。命题逻辑常转化为 CNF-SAT；等式逻辑可用重写系统；线性整数算术可用 Presburger 算法或 SMT；位向量可用 bit-blasting 转化为 SAT。

证明搜索过程输出两类结果：满足赋值，或不可满足证明。现代 SAT 求解器通常可输出 DRAT、LRAT 等可检查证明日志。

论证过程

DPLL 的正确性来自分裂完备性。对变量 x ，任意满足赋值要么令 $x=\text{true}$ ，要么令 $x=\text{false}$ 。因此搜索两个分支不会遗漏解。单子句传播保持等价的可满足性：若一个子句除某文字外都已为假，则该文字必须为真，否则该子句不满足。冲突回溯排除的是已经证明不可能满足的部分赋值。

若所有分支都冲突，则公式不可满足；若找到完整无冲突赋值，则公式可满足。

实现要点

DCA 的很多有限命题都可以交给 SAT/SMT。建议把证明器当作后端，把高层数学对象先编译为布尔、位向量或整数约束。为了开源可信，最好保存求解器版本、输入文件和可检查证明日志。

第43部分：全离散智能体概念

定义

这里的“全离散智能体”只是一个工程概念：一个感知、记忆、推理、规划和行动流程都用有限状态、整数张量、有限图、符号程序和可检查约束表达的系统。它不要求神秘的连续心智模型，也不宣称达到通用人工智能。

可把智能体状态写为：

$\text{AgentState} = \text{Memory} \times \text{Belief} \times \text{Goal} \times \text{Policy} \times \text{RuntimeState}$ 。

每个组件都应有有限编码。更新规则是：

$\text{AgentState}_{[t+1]} = F(\text{AgentState}_t, \text{Observation}_t)$ 。

论证过程

安全性和可控性的基本论证来自状态机不变式。若定义安全谓词 $\text{Safe}(s)$ ，并证明初始状态安全且每个允许动作都保持安全：

$\text{Safe}(s) \text{ AND } \text{action_allowed}(s,a) \rightarrow \text{Safe}(\text{step}(s,a))$ ，

则归纳可得任意有限执行前缀都安全。规划正确性也类似：若搜索算法只返回满足约束的计划，且执行器只执行已验证计划，则系统行为被限制在约束集合内。

实现要点

这个方向应保持克制。更实际的目标是构建可审计的整数推理组件、有限状态规划器、可验证安全约束和可复现评测。若涉及学习模块，应把学习得到的参数、推理过程和安全边界分开记录。

参考资料

1. Alan M. Turing, “On Computable Numbers, with an Application to the Entscheidungsproblem” , 1936. https://www.cs.virginia.edu/~robins/Turing_Paper_1936.pdf
2. Claude E. Shannon, “A Mathematical Theory of Communication” , Bell System Technical Journal, 1948. <https://people.math.harvard.edu/~ctm/home/text/others/shannon/entropy/entropy.pdf>
3. James W. Cooley and John W. Tukey, “An Algorithm for the Machine Calculation of Complex Fourier Series” , Mathematics of Computation, 1965. <https://web.stanford.edu/class/cme324/classics/cooley-tukey.pdf>
4. Mathieu Desbrun, Anil N. Hirani, Melvin Leok, Jerrold E. Marsden, “Discrete Exterior Calculus” , 2005. <https://arxiv.org/abs/math/0508341>
5. Keenan Crane, “Discrete Differential Geometry” course notes. <https://brickisland.net/ddg-web/>
6. Robin Forman, “A User’s Guide to Discrete Morse Theory” , 2002. <https://eudml.org/doc/123837>
7. Rocq Prover documentation, “Inductive types and recursive functions” . <https://rocq-prover.org/doc/V9.2.0/refman/language/core/inductive.html>
8. Benoit Jacob et al., “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference” , CVPR 2018. https://openaccess.thecvf.com/content_cvpr_2018/html/Jacob_Quantization_and_Training_CVPR_2018_paper.html
9. RISC-V Instruction Set Manual repository. <https://github.com/riscv/riscv-isa-manual>

10. NIST FIPS 203, Module-Lattice-Based Key-Encapsulation Mechanism Standard.
<https://csrc.nist.gov/pubs/fips/203/final>
11. Matthew Cook, “Universality in Elementary Cellular Automata” , Complex Systems, 2004.
https://www.complex-systems.com/abstracts/v15_i01_a01/
12. John von Neumann, “Theory of Self-Reproducing Automata” , 1966.
https://archive.org/details/theoryofselfrepr00vonn_0
13. Martin Davis, George Logemann, Donald Loveland, “A Machine Program for Theorem Proving” , Communications of the ACM, 1962. DOI: <https://doi.org/10.1145/368273.368557>
14. NIST, “FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard” , 2024.
<https://csrc.nist.gov/pubs/fips/203/final>
15. NIST, “FIPS 204: Module-Lattice-Based Digital Signature Standard” , 2024. <https://csrc.nist.gov/pubs/fips/204/final>
16. NIST, “FIPS 205: Stateless Hash-Based Digital Signature Standard” , 2024. <https://csrc.nist.gov/pubs/fips/205/final>
17. Benoit Jacob et al., “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference” , CVPR 2018.
https://openaccess.thecvf.com/content_cvpr_2018/html/Jacob_Quantization_and_Training_CVPR_2018_paper.html
18. LiteRT / TensorFlow Lite, “8-bit quantization specification” , 2024-2026 文档.
https://developers.google.com/edge/litert/conversion/tensorflow/quantization/quantization_spec
19. ONNX Runtime, “Quantize ONNX models” .
<https://onnxruntime.ai/docs/performance/model-optimizations/quantization.html>
20. IREE, “Intermediate Representation Execution Environment” . <https://iree.dev/>
21. RISC-V International, “The RISC-V Instruction Set Manual” . <https://github.com/riscv/riscv-isa-manual>
22. Sail ISA specification language. <https://github.com/rem-s-project/sail>
23. Project Everest, verified software stack for secure communication. <https://project-everest.github.io/>
24. HAcl^{*}, high-assurance cryptographic library. <https://hacl-star.github.io/>
25. Fiat-Crypto, formally verified cryptographic arithmetic generation. <https://github.com/mit-plv/fiat-crypto>
26. seL4 Foundation, verified microkernel project. <https://sel4.systems/>
27. CompCert, formally verified C compiler. <https://compcert.org/>
28. CakeML, verified implementation of ML. <https://cakeml.org/>
29. Rocq Prover documentation. <https://rocq-prover.org/doc/V9.2.0/refman/language/core/inductive.html>
30. Lean 4 / mathlib. <https://lean-lang.org/use-cases/mathlib/>
31. Isabelle theorem prover. <https://isabelle.in.tum.de/>
32. Dafny verification-aware programming language. <https://dafny.org/>
33. Kani Rust Verifier. <https://model-checking.github.io/kani/>
34. Z3 theorem prover. <https://github.com/Z3Prover/z3>
35. cvc5 SMT solver. <https://cvc5.github.io/>
36. Kissat SAT solver. <https://github.com/arminbiere/kissat>
37. CaDiCaL SAT solver. <https://github.com/arminbiere/cadical>
38. Google OR-Tools. <https://developers.google.com/optimization>
39. JuMP, Julia mathematical optimization modeling language. <https://jump.dev/JuMP.jl/stable/>
40. CVXPY, Python-embedded modeling language for convex optimization. <https://www.cvxpy.org/>
41. do-mpc, Python toolbox for model predictive control. <https://www.do-mpc.com/>

42. GUDHI, computational topology and topological data analysis library. <https://gudhi.inria.fr/index.html>
43. Ripser, persistent homology software. <https://github.com/Ripser/ripser>
44. Dionysus, computational topology package. <https://www.mrzv.org/software/dionysus/>
45. Guillaume Tauzin et al., “giotto-tda: A Topological Data Analysis Toolkit for Machine Learning and Data Exploration” , 2020. <https://arxiv.org/abs/2004.02551>
46. libigl, geometry processing library. <https://libigl.github.io/>
47. geometry-central, geometry processing library. <https://geometry-central.net/>
48. “Dxtr: A library for discrete exterior calculus” , Journal of Open Source Software, 2026. <https://www.theoj.org/joss-papers/joss.08110/10.21105.joss.08110.pdf>
49. SageMath, open-source mathematics software system. <https://www.sagemath.org/>
50. GAP, Groups, Algorithms, and Programming. <https://www.gap-system.org/>
51. FLINT, Fast Library for Number Theory. <https://flintlib.org/>
52. Macaulay2, software for algebraic geometry and commutative algebra. <https://macaulay2.com/>
53. Singular, computer algebra system for polynomial computations. <https://www.singular.uni-kl.de/>
54. arkworks algebra libraries. <https://github.com/arkworks-rs/algebra>
55. Winterfell STARK prover. <https://github.com/facebook/winterfell>
56. Plonky2 recursive SNARK/STARK library. <https://github.com/0xPolygonZero/plonky2>
57. AFF3CT, fast forward error correction toolbox. <https://aff3ct.github.io/>
58. NVIDIA Sionna, link-level and system-level simulations for communication systems. <https://developer.nvidia.com/sionna>
59. NetworkX, Python package for graph analysis. <https://networkx.org/>
60. SuiteSparse:GraphBLAS. <https://people.engr.tamu.edu/davis/GraphBLAS.html>
61. Qiskit SDK. <https://github.com/Qiskit/qiskit>
62. Cirq quantum computing framework. <https://quantumai.google/cirq>
63. QuTiP 5, Quantum Toolbox in Python. <https://arxiv.org/abs/2412.04705>
64. Stim stabilizer circuit simulator. <https://github.com/quantumlib/Stim>
65. Golly cellular automata explorer. <https://golly.sourceforge.io/>
66. TLA+ tools. <https://github.com/tlaplus/tlaplus>